

[200]FLOODSIM GIS PROJECT

Advanced Algorithm Reference

for the FloodEngine C++ Backend

Internal Technical Documentation

Version 2.0 · 2025

This document covers every algorithm layer of the FloodEngine DLL — from terrain preprocessing through hydrodynamic routing to ML-corrected depth prediction. Study each section before implementing changes to the C++ codebase.

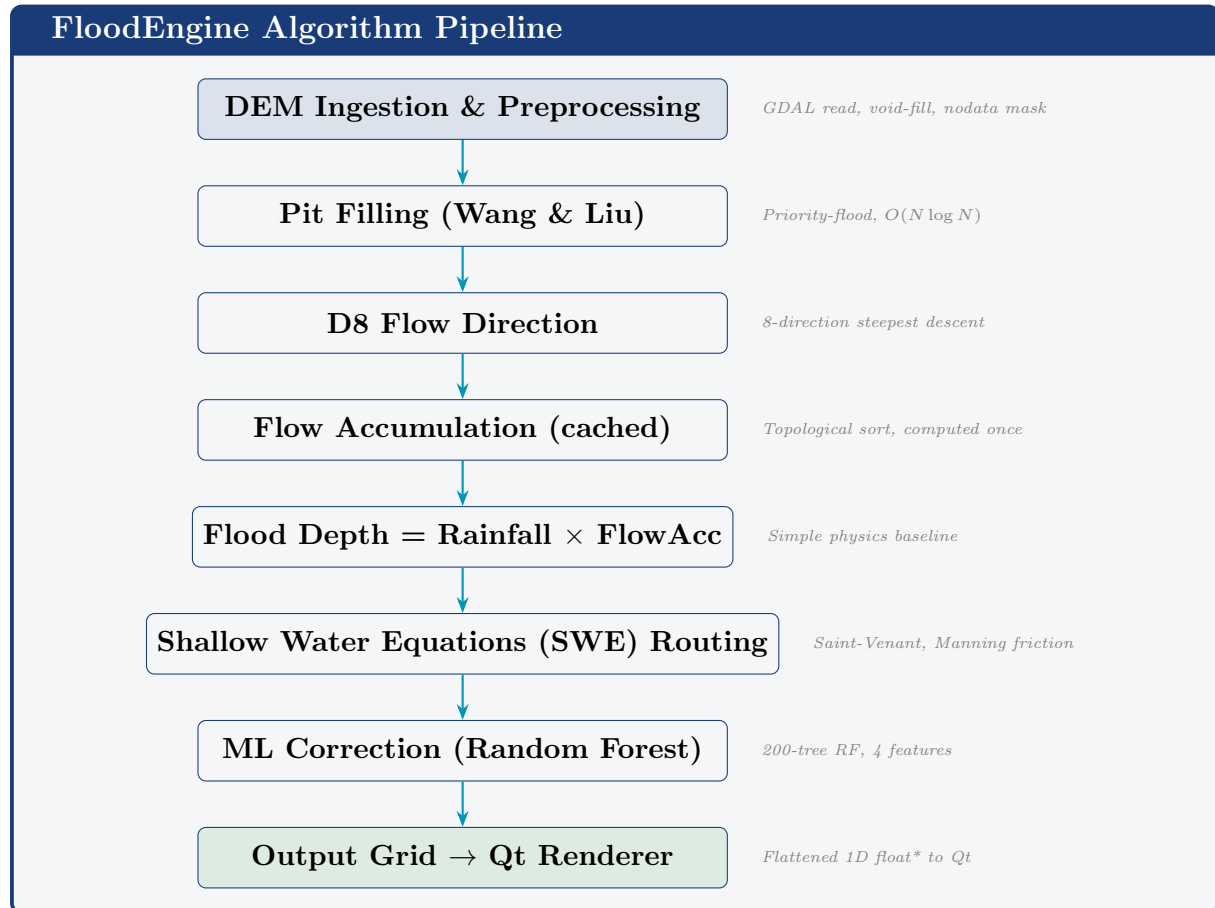
Contents

1	Overview and Algorithm Map	3
2	Terrain Preprocessing	3
2.1	Nodata Masking and Void Filling	3
2.2	Pit Filling — Wang & Liu Priority Flood	4
2.2.1	Mathematical Definition	4
2.2.2	Algorithm	5
3	Flow Direction Algorithms	5
3.1	D8 — Current Implementation Review	5
3.1.1	D8 Slope Calculation with Diagonal Correction	5
3.2	MFD — Multiple Flow Direction (Advanced)	6
3.2.1	Quinn et al. MFD Formulation	6
4	Flow Accumulation — Optimised Implementation	7
4.1	Current Problem	7
4.2	Topological Sort (Iterative, Cache-Friendly)	7
4.3	Complexity Comparison	8
5	Flood Depth Computation	8
5.1	Phase 1 Baseline: Rational Method	8
5.2	Manning’s Equation for Flow Velocity	9
5.3	Phase 2 Time Series Scaling (Qt-Side)	9
6	Shallow Water Equations (SWE) — Advanced Routing	10
6.1	1D Saint-Venant Equations	10
6.2	2D Diffusive Wave Approximation	10
6.3	Explicit Time Integration (Forward Euler)	11
6.3.1	CFL Stability Condition	11
7	Infiltration Models	11
7.1	Green-Ampt Model	11
8	Terrain Analysis Algorithms	12
8.1	Marching Squares — Enhanced Contour Lines	12
8.2	Curvature Analysis	12
8.3	TWI — Topographic Wetness Index	13
8.4	Watershed Delineation	13
9	Random Forest — ML Depth Correction	14
9.1	Current Architecture Review	14
9.2	Expanded Feature Vector (Recommended)	14
9.3	Tree Traversal (Current — Correct)	14
9.4	Physics-ML Blending	15

10 River Detection and Channel Extraction	15
10.1 Current Threshold Method	15
10.2 Area Threshold with Scale Correction	15
10.3 Channel Width Estimation	15
11 Boundary Condition Fix (Issue #3)	16
12 Implementation Roadmap for FloodSim	16
13 Glossary	17

1 Overview and Algorithm Map

The FloodEngine operates as a layered pipeline. Each layer transforms terrain or flow data into a richer representation. Understanding which algorithm feeds into which is essential before touching any C++ code.



Reading Guide

Each section is self-contained. The mathematical formulation comes first, then the algorithm pseudocode, then C++ implementation notes. You do not need to read sequentially — jump to whichever algorithm you are about to implement.

2 Terrain Preprocessing

Before any hydrological computation, the raw DEM must be conditioned. Raw DEMs contain artefacts — nodata voids, noise spikes, and most importantly **pits** (local minima) that break flow direction computation.

2.1 Nodata Masking and Void Filling

A nodata cell holds a sentinel value (commonly -9999 or `FLT_MAX`). These cells must be identified and either interpolated or excluded from all computations.

$$\text{valid}(i, j) = \begin{cases} \text{true} & \text{if } \text{dem}[i][j] \neq \text{NODATA} \\ \text{false} & \text{otherwise} \end{cases}$$

Bilinear interpolation for isolated voids:

$$z(i, j) = \frac{\sum_{(p,q) \in \mathcal{N}(i,j)} w_{pq} \cdot z(p, q)}{\sum_{(p,q) \in \mathcal{N}(i,j)} w_{pq}}, \quad w_{pq} = \frac{1}{d(i, j, p, q)^2}$$

2.2 Pit Filling — Wang & Liu Priority Flood

Why This Matters

Your current D8 implementation returns -1 for local minima (pits). On a real DEM, 5–15% of cells can be pits. Every pit breaks flow accumulation for all cells draining into it. Pit filling is non-negotiable for production flood simulation.

The Wang & Liu (2006) algorithm is the industry standard. It guarantees every cell has a downhill path to the DEM boundary using a min-heap priority queue.

2.2.1 Mathematical Definition

Given elevation grid Z , produce filled grid Z' such that:

$$Z'(i, j) \geq Z(i, j) \quad \forall (i, j)$$

$$\forall (i, j) \exists \text{ monotone decreasing path from } (i, j) \text{ to a boundary cell in } Z'$$

2.2.2 Algorithm

Algorithm 1 Wang & Liu Priority Flood Pit Filling

Require: DEM grid $Z[R][C]$, boundary cells $\mathcal{B}$

Ensure: Filled DEM $Z'[R][C]$, no internal pits

```

1  $Z' \leftarrow Z$ 
2  $open \leftarrow$  empty min-heap (priority = elevation)
3  $closed \leftarrow$  boolean grid, all false
4 for all boundary cells  $(i, j) \in \mathcal{B}$  do
5   push  $(Z'[i][j], i, j)$  into  $open$ 
6    $closed[i][j] \leftarrow \mathbf{true}$ 
7 end for
8 while  $open$  is not empty do
9    $(e, i, j) \leftarrow$  pop minimum from  $open$ 
10  for all valid 8-neighbours  $(p, q)$  of  $(i, j)$  do
11    if  $closed[p][q]$  then continue
12    end if
13     $Z'[p][q] \leftarrow \max(Z'[p][q], e)$  // raise pit floor
14    push  $(Z'[p][q], p, q)$  into  $open$ 
15     $closed[p][q] \leftarrow \mathbf{true}$ 
16  end for
17 end while
18 return  $Z'$ 

```

Complexity

Time: $O(N \log N)$ where $N = R \times C$. Space: $O(N)$. This replaces your current `-1` return in `getFlowDirection()` — after pit filling, every cell has a valid downhill neighbour.

3 Flow Direction Algorithms

3.1 D8 — Current Implementation Review

D8 (Deterministic 8-direction) assigns each cell a single drainage direction — the steepest downhill neighbour. Your current implementation is correct but limited.

3.1.1 D8 Slope Calculation with Diagonal Correction

A key improvement your current code likely lacks: diagonal neighbours are **farther away** than cardinal neighbours. Without distance correction, diagonal gradients are overestimated.

Cardinal distance: $d_c = \Delta x$ (cell resolution in metres)

Diagonal distance: $d_d = \sqrt{2} \Delta x$

Corrected gradient for direction k :

$$s_k(i, j) = \frac{Z(i, j) - Z(i + \delta_i^k, j + \delta_j^k)}{d^k}$$

$$d^k = \begin{cases} \Delta x & k \in \{E, W, N, S\} \\ \sqrt{2} \Delta x & k \in \{NE, NW, SE, SW\} \end{cases}$$

$$\text{flowDir}(i, j) = \arg \max_k s_k(i, j) \quad [\text{only if } s_k > 0]$$

3.2 MFD — Multiple Flow Direction (Advanced)

D8 forces all flow into one cell. Real terrain spreads flow across multiple downhill cells. MFD is essential for accurate flood area prediction in flat zones.

3.2.1 Quinn et al. MFD Formulation

For each downhill neighbour k of cell (i, j) :

$$f_k(i, j) = \frac{\max(s_k, 0)^p}{\sum_{k: s_k > 0} \max(s_k, 0)^p}$$

where p is a weighting exponent (typically $p = 1$ for linear, $p = 2$ for quadratic partitioning).

Flow accumulation then becomes:

$$A(i, j) = 1 + \sum_{(p, q) \rightarrow (i, j)} f_{pq \rightarrow ij} \cdot A(p, q)$$

Algorithm 2 MFD Flow Fraction Computation**Require:** Filled DEM Z' , exponent p **Ensure:** Flow fraction matrix $F[i][j][k]$ for 8 directions

```

1  for all cells  $(i, j)$  do
2       $sum \leftarrow 0$ 
3      for all valid neighbours  $k$  do
4           $s_k \leftarrow (Z'[i][j] - Z'[\text{nbr}]) / d^k$ 
5          if  $s_k > 0$  then
6               $F[i][j][k] \leftarrow s_k^p$ 
7               $sum += s_k^p$ 
8          else
9               $F[i][j][k] \leftarrow 0$ 
10         end if
11     end for
12     if  $sum > 0$  then
13         for all  $k$  do
14              $F[i][j][k] /= sum$  // normalise to probabilities
15         end for
16     end if
17 end for

```

4 Flow Accumulation — Optimised Implementation

4.1 Current Problem

Your architecture doc (Issue #2) identifies that `computeFlowAccumulation()` is called every time `FloodDepthGridWithRain` runs. This is an $O(N)$ recursive computation that should run exactly once after `loadFromGrid()`.

4.2 Topological Sort (Iterative, Cache-Friendly)

The recursive DFS in your current code has two problems: stack overflow risk on large DEMs (>10M cells), and poor cache locality. The correct approach is a topological sort using in-degree counting.

Key insight: Flow accumulation is a DAG (directed acyclic graph) computation. Process cells in reverse topological order — upstream cells first, then accumulate downstream.

$$\text{in_degree}(i, j) = |\{(p, q) : \text{flowDir}(p, q) \rightarrow (i, j)\}|$$

$$A(i, j) = 1 + \sum_{(p, q) : \text{flowDir}(p, q) = (i, j)} A(p, q)$$

Algorithm 3 Iterative Flow Accumulation via Topological Sort**Require:** Flow direction grid $D[R][C]$ **Ensure:** Flow accumulation grid $A[R][C]$, cached as member

```

1  $A[i][j] \leftarrow 1 \forall (i, j)$ 
2 Compute  $\text{in\_degree}[i][j]$  for all cells
3  $Q \leftarrow$  queue of all cells with  $\text{in\_degree} = 0$ 
4 while  $Q$  not empty do
5    $(i, j) \leftarrow Q.\text{pop}()$ 
6    $(p, q) \leftarrow \text{nextCell}(i, j, D[i][j])$ 
7   if  $(p, q)$  is valid then
8      $A[p][q] += A[i][j]$ 
9      $\text{in\_degree}[p][q] -= 1$ 
10    if  $\text{in\_degree}[p][q] = 0$  then
11       $Q.\text{push}(p, q)$ 
12    end if
13  end if
14 end while
15 return  $A$ 

```

// store as cachedFlowAcc member variable

C++ Implementation Note

In `Raster.h`, add member: `std::vector<std::vector<double>>` `cachedFlowAcc`; and `bool flowAccCached = false`; . Call the topological sort inside `loadFromGrid()` and set `flowAccCached = true`. `FloodDepthGridFinal()` reads `cachedFlowAcc` directly.

4.3 Complexity Comparison

Method	Time	Space	Stack Safe
Current recursive DFS	$O(N)$	$O(N)$ depth	No (overflow $\geq 1\text{M}$ cells)
Iterative topological	$O(N)$	$O(N)$ heap	Yes
Cached (after fix)	$O(1)$ per query	same	Yes

5 Flood Depth Computation**5.1 Phase 1 Baseline: Rational Method**

Your current formula $\text{depth}[i][j] = \text{rainfall} \times A(i, j)$ is a simplified rational method. It is a useful baseline but physically incomplete. The full rational method is:

$$Q(i, j) = C \cdot I \cdot A(i, j) \cdot \Delta x^2$$

$$\text{depth}(i, j) = \frac{Q(i, j)}{v(i, j) \cdot w}$$

where:

- C = runoff coefficient (0.3–0.9, terrain-dependent)

- I = rainfall intensity (mm/hr)
- $A(i, j)$ = flow accumulation (number of upstream cells)
- Δx^2 = cell area in m^2
- $v(i, j)$ = flow velocity from Manning's equation (m/s)
- w = channel width (m), or set to Δx for overland flow

5.2 Manning's Equation for Flow Velocity

Manning's equation gives velocity from slope and hydraulic radius:

$$v(i, j) = \frac{1}{n} \cdot R_h^{2/3} \cdot S^{1/2}$$

where:

- n = Manning's roughness coefficient (0.01–0.15, see table below)
- R_h = hydraulic radius $\approx$ depth (for shallow overland flow)
- S = local slope magnitude = $\max_k s_k(i, j)$

Roughness values n for FloodSim:

Surface	n
Concrete channel	0.013
Natural river	0.035
Floodplain grassland	0.060
Dense forest	0.120
Urban (buildings)	0.080

5.3 Phase 2 Time Series Scaling (Qt-Side)

Per your architecture document, the engine outputs a single final grid. Qt generates time steps by scaling:

$$\text{depth}_t(i, j) = \text{depth}_{\text{final}}(i, j) \times f(t)$$

Recommended hydrograph factors (replace your linear 10%–100%):

Hour t	Factor $f(t)$	Model
1	0.05	Infiltration dominates
2	0.18	
3	0.42	
4	0.72	
5	0.90	Peak approaching
6	1.00	Peak flood
7	0.88	Recession
8	0.65	
9	0.38	
10	0.15	Near-normal

The S-curve shape follows: $f(t) = \frac{1}{1+e^{-k(t-t_{\text{peak}})}}$ where k controls steepness.

6 Shallow Water Equations (SWE) — Advanced Routing

The Shallow Water Equations (also called Saint-Venant equations) are the physically correct description of flood propagation. They replace the static depth formula with a dynamic simulation.

When to Use SWE

SWE is computationally expensive ($\sim 50\times$ slower than your current method). Implement it as an optional high-accuracy mode, selectable from Qt. Use the rational method as default, SWE for detailed analysis exports.

6.1 1D Saint-Venant Equations

Continuity equation (mass conservation):

$$\frac{\partial h}{\partial t} + \frac{\partial(uh)}{\partial x} = r - i$$

Momentum equation:

$$\frac{\partial(uh)}{\partial t} + \frac{\partial}{\partial x} \left(u^2h + \frac{1}{2}gh^2 \right) = gh(S_0 - S_f)$$

where:

- $h(x, t)$ = water depth (m)
- $u(x, t)$ = flow velocity (m/s)
- r = rainfall rate (m/s)
- i = infiltration rate (Green-Ampt model, m/s)
- $g = 9.81 \text{ m/s}^2$
- S_0 = bed slope
- $S_f = n^2u^2/h^{4/3}$ = friction slope (Manning)

6.2 2D Diffusive Wave Approximation

The full 2D SWE is impractical for real-time desktop use. The **diffusive wave** approximation drops the inertia terms and remains accurate for sub-critical flow (most floodplain scenarios):

$$\frac{\partial h}{\partial t} = r - i - \nabla \cdot \mathbf{q}$$

$$\mathbf{q} = -\frac{h^{5/3}}{n} \cdot \frac{\nabla(h + Z)}{|\nabla(h + Z)|^{1/2}} \cdot \text{sign}(\nabla(h + Z))$$

6.3 Explicit Time Integration (Forward Euler)

Algorithm 4 2D Diffusive Wave Time Stepping

Require: DEM Z , Manning n , rainfall r , total time T , timestep Δt

Ensure: Water depth grid $h[R][C]$ at each timestep

```

1  $h[i][j] \leftarrow 0 \forall (i, j)$  // initial dry conditions
2  $\Delta t \leftarrow$  CFL condition // see below
3 for each timestep  $t = 0, \Delta t, 2\Delta t, \dots, T$  do
4   for all interior cells  $(i, j)$  do
5     Compute water surface  $W[i][j] = h[i][j] + Z[i][j]$ 
6   end for
7   for all interior cells  $(i, j)$  do
8     for all 4 cardinal neighbours  $(p, q)$  do
9        $S_{pq} \leftarrow (W[i][j] - W[p][q]) / \Delta x$ 
10       $h_{\text{face}} \leftarrow \max(h[i][j], h[p][q])$ 
11       $q_{pq} \leftarrow \frac{h_{\text{face}}^{5/3}}{n} \cdot \sqrt{|S_{pq}|} \cdot \text{sign}(S_{pq})$ 
12    end for
13     $h_{\text{new}}[i][j] \leftarrow h[i][j] + \Delta t \left( r - i - \frac{\sum q_{\text{out}} - \sum q_{\text{in}}}{\Delta x} \right)$ 
14     $h_{\text{new}}[i][j] \leftarrow \max(h_{\text{new}}[i][j], 0)$  // no negative depth
15  end for
16   $h \leftarrow h_{\text{new}}$ 
17  Emit  $h$  to Qt playback buffer at this timestep
18 end for
```

6.3.1 CFL Stability Condition

$$\Delta t \leq \text{CFL} \cdot \frac{\Delta x}{\sqrt{g \cdot h_{\max}}}$$

Typically CFL = 0.5. In C++:

$$\Delta t = 0.5 \cdot \frac{\Delta x}{\sqrt{9.81 \cdot \max(h)}} \quad [\text{recompute each step}]$$

7 Infiltration Models

Infiltration reduces effective rainfall. Without it, every raindrop becomes runoff — which severely overestimates flood depth in areas with permeable soil.

7.1 Green-Ampt Model

The Green-Ampt model is the standard in engineering hydrology and is physically based (piston-flow through soil):

Infiltration rate at time t :

$$f(t) = K_s \left(1 + \frac{\psi \cdot \Delta\theta}{F(t)} \right)$$

Cumulative infiltration $F(t)$ (implicit, solved iteratively):

$$F(t) = K_s \cdot t + \psi \cdot \Delta\theta \cdot \ln \left(1 + \frac{F(t)}{\psi \cdot \Delta\theta} \right)$$

where:

- K_s = saturated hydraulic conductivity (mm/hr), soil-dependent
- ψ = wetting front capillary pressure (mm)
- $\Delta\theta = \theta_s - \theta_i$ = available soil moisture

Soil parameter table for FloodSim:

Soil type	K_s (mm/hr)	ψ (mm)	$\Delta\theta$
Sand	117.8	49.5	0.417
Silt loam	6.8	166.8	0.486
Clay	0.3	316.3	0.385
Urban/impervious	0.0	–	0.0

8 Terrain Analysis Algorithms

8.1 Marching Squares — Enhanced Contour Lines

Your existing Marching Squares implementation handles contour extraction. The key upgrade is **linear interpolation** along cell edges (vs. binary threshold), which eliminates staircasing artefacts.

For a contour at level c crossing the edge between cells with values z_0 and z_1 :

$$t = \frac{c - z_0}{z_1 - z_0}, \quad 0 \leq t \leq 1$$

The contour point lies at parametric position t along the edge. This is the standard improvement over your binary Marching Squares.

8.2 Curvature Analysis

Curvature identifies ridges (flow divergence) and channels (flow convergence). Essential for accurate watershed boundary detection beyond simple D8.

Plan curvature (horizontal, affects flow divergence):

$$\kappa_{\text{plan}}(i, j) = \frac{-1}{(p^2 + q^2)^{3/2}} [r p^2 + 2s p q + t q^2]$$

Profile curvature (vertical, affects flow acceleration):

$$\kappa_{\text{prof}}(i, j) = \frac{-1}{(p^2 + q^2)(1 + p^2 + q^2)^{3/2}} [rp^2 + 2spq + tq^2]$$

where, using central finite differences on the DEM grid Z :

$$p = \frac{Z_{i,j+1} - Z_{i,j-1}}{2\Delta x}, \quad q = \frac{Z_{i-1,j} - Z_{i+1,j}}{2\Delta y}$$

$$r = \frac{Z_{i,j+1} - 2Z_{i,j} + Z_{i,j-1}}{\Delta x^2}, \quad s = \frac{Z_{i-1,j+1} - Z_{i-1,j-1} - Z_{i+1,j+1} + Z_{i+1,j-1}}{4\Delta x\Delta y}$$

$$t = \frac{Z_{i-1,j} - 2Z_{i,j} + Z_{i+1,j}}{\Delta y^2}$$

Interpretation: $\kappa < 0$ = convergent (channel), $\kappa > 0$ = divergent (ridge).

8.3 TWI — Topographic Wetness Index

TWI is the single most important derived terrain variable for flood susceptibility. It quantifies where water naturally accumulates:

$$\text{TWI}(i, j) = \ln\left(\frac{A(i, j) \cdot \Delta x^2}{\tan(\beta(i, j))}\right)$$

where $\beta(i, j)$ is the local slope angle:

$$\beta(i, j) = \arctan\left(\sqrt{p^2 + q^2}\right)$$

High TWI (> 10) $\Rightarrow$ high flood probability. Use as a feature in your RF model and for risk marker placement in Qt.

8.4 Watershed Delineation

Watershed boundaries are determined by tracing flow directions from outlet points. The algorithm identifies all cells that drain to a given outlet.

Algorithm 5 Watershed Delineation by Upstream Tracing**Require:** Flow direction grid D , outlet cell (r_0, c_0) **Ensure:** Binary watershed mask $W[R][C]$

```

1  $W[i][j] \leftarrow 0 \forall (i, j)$ 
2  $queue \leftarrow \{(r_0, c_0)\}$ 
3  $W[r_0][c_0] \leftarrow 1$ 
4 while  $queue$  not empty do
5    $(i, j) \leftarrow queue.pop()$ 
6   for all 8-neighbours  $(p, q)$  of  $(i, j)$  do
7     if  $W[p][q] = 0$  and  $nextCell(p, q, D[p][q]) = (i, j)$  then
8        $W[p][q] \leftarrow 1$  // (p, q) drains into (i, j)
9        $queue.push(p, q)$ 
10    end if
11  end for
12 end while
13 return  $W$ 

```

9 Random Forest — ML Depth Correction

9.1 Current Architecture Review

Your RF uses 4 features: $[rain_t, rain_{t-1}, A(i, j), Z(i, j)]$. This is a solid baseline. The following extensions will significantly improve prediction accuracy.

9.2 Expanded Feature Vector (Recommended)

Index	Feature	Rationale
0	$rain_t$	Current rainfall
1	$rain_{t-1}$	Antecedent wetness
2	$A(i, j)$	Upstream drainage area
3	$Z(i, j)$	Elevation
4	$TWI(i, j)$	Topographic wetness
5	$\kappa_{plan}(i, j)$	Flow convergence
6	$\beta(i, j)$	Local slope
7	$dist_river(i, j)$	Distance to nearest river cell

9.3 Tree Traversal (Current — Correct)

For a single tree T with nodes $\mathcal{N}$, prediction for feature vector $\mathbf{x}$:

$$\text{predict}_T(\mathbf{x}) = \text{leaf value reached by: } \text{node} \leftarrow \begin{cases} \text{left child} & x_{f_{\text{node}}} \leq \theta_{\text{node}} \\ \text{right child} & \text{otherwise} \end{cases}$$

Ensemble prediction (average of 200 trees):

$$\hat{y}(i, j) = \frac{1}{200} \sum_{k=1}^{200} \text{predict}_{T_k}(\mathbf{x}_{ij})$$

9.4 Physics-ML Blending

Rather than using ML output directly, blend with physics baseline for robustness:

$$\text{depth}_{\text{final}}(i, j) = \alpha \cdot \hat{y}_{\text{ML}}(i, j) + (1 - \alpha) \cdot \text{depth}_{\text{physics}}(i, j)$$

Set $\alpha = 0.7$ (70% ML, 30% physics). This prevents ML from producing physically impossible values (negative depth, depth exceeding rainfall volume).

10 River Detection and Channel Extraction

10.1 Current Threshold Method

Your current implementation marks cells as river if $A(i, j) \geq 3$. This threshold is dimensionless and scale-dependent. On a 10m DEM vs. a 30m DEM, it produces completely different rivers.

10.2 Area Threshold with Scale Correction

$$\text{isRiver}(i, j) = \begin{cases} 1 & A(i, j) \cdot \Delta x^2 \geq A_{\min} \\ 0 & \text{otherwise} \end{cases}$$

where $A_{\min}$ is the minimum drainage area in m^2 to define a channel. Typical values:

- Small ephemeral channels: $A_{\min} = 1,000 \text{ m}^2$
- Perennial streams: $A_{\min} = 10,000 \text{ m}^2$
- Rivers: $A_{\min} = 100,000 \text{ m}^2$

10.3 Channel Width Estimation

Leopold's empirical power law for channel width from drainage area:

$$W_{\text{channel}}(i, j) = a \cdot (A(i, j) \cdot \Delta x^2)^b$$

Typical coefficients: $a = 1.22$, $b = 0.50$ (humid temperate climates). This gives channel width in metres and can be used to modulate flood spread in your OpenGL shader.

11 Boundary Condition Fix (Issue #3)

Fix for OutflowInflow Boundary Bug

Your architecture doc identifies that `nextCell()` returns $(-1, -1)$ for border cells, and `OutflowInflow` does not bounds-check before indexing. The fix:

Algorithm 6 Corrected OutflowInflow with Boundary Guard

```

1 for all cells  $(i, j)$  do
2    $p \leftarrow \text{nextCell}(i, j, D[i][j])$ 
3   if  $p.\text{first} < 0$  or  $p.\text{first} \geq \text{rows}$  then continue
4   end if
5   if  $p.\text{second} < 0$  or  $p.\text{second} \geq \text{cols}$  then continue
6   end if
7    $\text{outflow} \leftarrow \text{depth}[i][j] \times 0.3$ 
8    $\text{depth}[p.\text{first}][p.\text{second}] += \text{outflow}$ 
9    $\text{depth}[i][j] -= \text{outflow}$ 
10 end for
```

The two `continue` guards are the complete fix for Issue #3.

12 Implementation Roadmap for FloodSim

Phase	Task	Component	Priority
P1	Fix boundary bug (Issue #3)	<code>Raster.cpp</code>	Critical
P1	Remove fake t parameter	<code>Raster.cpp</code>	High
P1	Cache flow accumulation	<code>Raster.h/.cpp</code>	High
P2	Wang & Liu pit filling	<code>Raster.cpp</code>	High
P2	D8 diagonal distance correction	<code>Raster.cpp</code>	Medium
P2	TWI computation	<code>Raster.cpp</code>	Medium
P2	Scale-corrected river detection	<code>Raster.cpp</code>	Medium
P3	MFD flow direction	<code>Raster.cpp</code>	Medium
P3	Green-Ampt infiltration	<code>Raster.cpp</code>	Medium
P3	Expanded RF feature vector	<code>train_rf.py</code> + <code>RF.cpp</code>	Medium
P4	Diffusive wave SWE	<code>SWESolver.cpp</code> (new)	Low
P4	Physics-ML blending	<code>FloodEngineAPI.cpp</code>	Low

Key Principle

P1 fixes make the existing engine correct. P2 makes it production-grade for real DEMs. P3 makes it scientifically accurate. P4 makes it a research-quality tool. Build P1 first — do not add P3 features onto a broken P1 engine.

13 Glossary

CFL	Courant-Friedrichs-Lewy — numerical stability condition for explicit time integration
D8	Deterministic 8-direction — single-direction flow routing
DEM	Digital Elevation Model — raster grid of terrain elevations
DAG	Directed Acyclic Graph — structure of flow direction connections
MFD	Multiple Flow Direction — distributed flow routing
SWE	Shallow Water Equations — 2D Saint-Venant equations for flood dynamics
TWI	Topographic Wetness Index — terrain-derived flood susceptibility measure
RF	Random Forest — ensemble ML model for depth correction
Manning's n	Empirical roughness coefficient for open channel flow
Green-Ampt	Physically-based soil infiltration model

FloodSim GIS · Internal Technical Documentation

Advanced Algorithm Reference · Version 2.0 · 2025

C++ / Qt6 / OpenGL 3.3 / GDAL